QuillAudits

AUDIT REPORT

March 2026

For

Table of Content

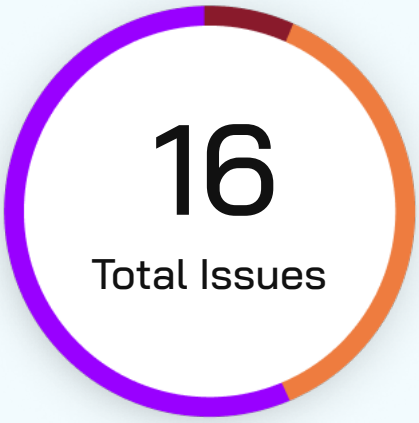
Executive Summary

Project Name	LifeSol
Overview	This Solana Anchor program implements a staking protocol with multi-level referral rewards and strict vault reservation accounting. When a user stakes, the contract calculates staking rewards, up to 10 levels of referral commissions, and a 5% market fee, then ensures the vault has sufficient unreserved balance before proceeding. A unique Stake PDA is derived per user using a nonce, created via CPI, and initialized manually with a discriminator and serialized data. Tokens are transferred from the user to the vault, the market fee is distributed, and the total reserved amount is tracked in config to prevent over-allocation. Referral rewards are propagated through a verified PDA chain using remaining_accounts, updating each referrer's accumulated rewards and active referral counts. Reward claiming is time-proportional: users can claim accrued staking rewards incrementally, claim referral rewards based on structured eligibility thresholds, or withdraw the full stake amount (plus remaining rewards) after the staking period ends.
Source Code link	Zip File was Provided for Audit
Branch	main
Contracts in Scope	Lib.rs
Blockchain	Solana
Method	Manual review, Functional testing, Automated tests
Review 1	25th February 2026 - 3rd March 2026
Updated Code Received	5th March 2026
Review 2	5th March 2026 - 10th March 2026
Fixed In	branch: Dev **6a2b367a137648c41ac39429f162f4f6bbf9c843**

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	1 (6.25%)
High	0 (0.0%)
Medium	6 (37.50%)
Low	0 (0.0%)
Informational	9 (56.25%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	0	0
Partially Resolved	0	0	0	0	0
Resolved	**1**	0	**6**	0	**9**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Missing ownership validation in claim_ref_rewards allows anyone to steal referral rewards	Critical	Resolved
2	Predictable stake PDA address allows griefing attack via lamport donation and hence preventing genuine users from staking again	Medium	Resolved
3	config.reserved incorrectly includes already-transferred market fee, causing inflated reservation and denial of service on subsequent stakes	Medium	Resolved
4	All stake calculations use pre-fee amount, inflating rewards, referral Payouts, and reservations while effectively bypassing the market fee	Medium	Resolved
5	Malicious referees can grief their top level referrers by not providing their accounts in `remaining_accounts` and hence preventing them from receiving ref rewards	Medium	Resolved
6	break used instead of continue in claim_ref_rewards, permanently blocking higher-level reward claims	Medium	Resolved
7	Initialization can be frontrun, granting attacker full control over pre-funded vault	Medium	Resolved

Issue No.	Issue Title	Severity	Status
10	No admin transfer functionality	Informational	Resolved
11	No pause functionality	Informational	Resolved
12	No functionality to change the market token account	Informational	Resolved
13	Market token account not validated during initialization	Informational	Resolved
14	admin Parameter and admin_pubkey Account Can Differ During Initialization	Informational	Resolved
15	admin Parameter and admin_pubkey Account Can Differ During Initialization	Informational	Resolved
16	RewardsDistributor.sweep() can remove tokens still owed to users	Informational	Resolved
17	initialize() derives PDA bumps via find_program_address instead of using ctx.bumps	Informational	Resolved

Checked Vulnerabilities

We have scanned the solana program for commonly known and more specific vulnerabilities. Here are some of the commonly known vulnerabilities that we considered:

- Signer authorization
- Account data matching
- Sysvar address checking
- Owner checks
- Type cosplay
- Initialization
- Arbitrary cpi
- Duplicate mutable accounts
- Bump seed canonicalization
- PDA Sharing
- Incorrect closing accounts
- Missing rent exemption checks
- Arithmetic overflows/underflows
- Numerical precision errors
- Solana account confusions
- Casting truncation
- Insufficient SPL token account verification
- Signed invocation of unverified programs

Techniques and Methods

Throughout the audit of Solana Programs, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Critical Severity Issues

Missing ownership validation in claim_ref_rewards allows anyone to steal referral rewards

Resolved

Path

programs/vault/src/instructions/admin/add_token.rs

Description

The ClaimRefCtx account struct does not validate that the user_stats account belongs to the user signer. Any arbitrary signer can pass a victim's user_stats account and their own user_token_account to drain the victim's referral rewards.

Vulnerable account struct:

```
#[derive(Accounts)]
pub struct ClaimRefCtx<'info> {
    #[account(mut)]
    pub config: Account<'info, Config>,

    #[account(mut)]                    // ← No seeds, no has_one, no ownership check
    pub user_stats: Account<'info, UserStats>,

    #[account(mut)]
    pub user: Signer<'info>,           // ← Just needs to sign, no tie to user_stats

    #[account(
        mut,
        constraint = user_token_account.mint == config.token_mint,
        // ← No check that this account belongs to `user`
    )]
    pub user_token_account: InterfaceAccount<'info, TokenAccount>,
    // ...
}
```

Impact

Anyone can pass others' userStats accounts to claim ref rewards.

Recommendation

Bind user_stats to the user signer using PDA seeds and add ownership validation on user_token_account:

```
#[derive(Accounts)]
pub struct ClaimRefCtx<'info> {
    #[account(mut, seeds = [b"config"], bump)]
    pub config: Account<'info, Config>,

    #[account(
        mut,
        seeds = [b"user_stats", user.key().as_ref()], // ← Binds to signer
        bump,
    )]
    pub user_stats: Account<'info, UserStats>,

    #[account(mut)]
    pub user: Signer<'info>,

    #[account(
        mut,
        constraint = user_token_account.mint == config.token_mint,
        constraint = user_token_account.owner == user.key() // ← Ensures funds go to signer
    )]
    pub user_token_account: InterfaceAccount<'info, TokenAccount>,

    #[account(mut, address = config.vault)]
    pub vault: InterfaceAccount<'info, TokenAccount>,

    #[account(
        seeds = [b"vault_authority", config.key().as_ref()],
        bump
    )]
    /// CHECK: PDA authority for vault
    pub vault_authority: UncheckedAccount<'info>,

    pub token_program: Program<'info, Token2022>,
}
```

Medium Severity Issues

Predictable stake PDA address allows griefing attack via lamport donation and hence preventing genuine users from staking again

Resolved

Path

programs/vault/src/instructions/admin/init.rs
programs/withdrawal-approver/src/instructions/admin/init.rs

Description

The stake function creates the stake account using system_instruction::create_account via invoke_signed. This instruction requires the target account to have zero lamports and not exist at the time of invocation. Since the stake PDA address is fully deterministic (derived from known public seeds), any attacker can pre-compute it and send a small dust amount (e.g., 1 lamport) to that address before the user's transaction executes.

When create_account is later called on an already-funded address, the Solana runtime rejects it - permanently blocking that nonce's stake PDA.

Vulnerable code:

```
let (stake_pda, stake_bump) = Pubkey::find_program_address(
    &[b"stake", user_key.as_ref(), &current_nonce.to_le_bytes()],
    ctx.program_id,
);

// This will fail if stake_pda already has any lamports
let create_account_ix = anchor_lang::solana_program::system_instruction::create_account(
    &user_key,
    &stake_pda,
    lamports,
    stake_space as u64,
    ctx.program_id,
);

anchor_lang::solana_program::program::invoke_signed(
    &create_account_ix,
    &[
        ctx.accounts.user.to_account_info(),
        ctx.accounts.stake.to_account_info(),
        ctx.accounts.system_program.to_account_info(),
    ],
    signer,
)?;
```

Impact

- Attacker pre-computes victim's stake PDA using ("stake", victim_pubkey, nonce_bytes)
  Attacker sends 1 lamport to that address (costs almost nothing)
- Victim's stake() call fails with a system program error on create_account
- Since stake_nonce is only incremented after successful account creation, the nonce does not advance — the victim is permanently stuck on that nonce
- Every subsequent stake attempt for the same nonce is also blocked, permanently DoS-ing the victim's ability to stake
- This cascades to the referral system: a blocked user cannot stake, which means they cannot add referrals, locking their entire referral subtree out of rewards

Recommendation

Replace manual create_account with Anchor's native init constraint on the stake account, which uses allocate + assign + transfer under the hood and handles pre-existing accounts gracefully:

```
#[derive(Accounts)]
#[instruction(stake_nonce: u64, amount: u64, period: i64)]
pub struct StakeCtx<'info> {
    #[account(mut)]
    pub config: Account<'info, Config>,

    #[account(
        init,
        payer = user,
        seeds = [b"stake", user.key().as_ref(), &stake_nonce.to_le_bytes()],
        bump,
        space = 8 + 32 + 8 + 8 + 8 + 8 + 1 + 8
    )]
    pub stake: Account<'info, Stake>,  // Typed, not UncheckedAccount

    #[account(mut)]
    pub user: Signer<'info>,
    // ...existing code...
}
```

config.reserved incorrectly includes already-transferred market fee, causing inflated reservation and denial of service on subsequent stakes

Resolved

Path

programs/vault/src/instructions/admin/grant_role.rs
programs/withdrawal-approver/src/instructions/admin/grant_role.rs

Description

In the stake() instruction, the market fee (5% of staked amount) is calculated and immediately transferred out of the vault to the market account:

```

// market_fee is transferred OUT of the vault immediately
spl_token::transfer(cpi_ctx_market, market_fee)?;
```

However, market_fee is also added into total_reserved, which is then accumulated into config.reserved:

```
let total_reserved = amount
    .checked_add(max_reward)?
    .checked_add(total_ref_rewards)?
    .checked_add(market_fee)?;   // <-- market_fee already left the vault

ctx.accounts.config.reserved = ctx.accounts.config.reserved
    .checked_add(total_reserved)?;
```

This means config.reserved tracks tokens that no longer exist in the vault, inflating the reserved balance by market_fee on every single stake. When the next staker calls stake(), the available balance check becomes:

```
let available_balance = ctx.accounts.vault.amount
    .saturating_sub(ctx.accounts.config.reserved); // reserved is inflated
require!(
    available_balance >= total_reserved,
    CustomError::InsufficientVaultBalance
);
```

Since config.reserved is inflated by the cumulative sum of all past market fees, available_balance is understated, causing the check to fail even when the vault holds sufficient real funds. This causes every subsequent stake after the first to revert with InsufficientVaultBalance.

Impact

After the first stake, all subsequent stake() calls will revert with InsufficientVaultBalance because config.reserved overstates the committed balance by the accumulated market fees of all prior stakes.

Recommendation

Exclude market_fee from total_reserved. The fee is transferred out of the vault immediately during stake() and therefore should never be tracked as a future obligation. config.reserved should only account for obligations the vault still needs to pay out: the staked principal (`amount`), the staking rewards (`max_reward`), and the referral rewards (`total_ref_rewards`).

```
// Remove market_fee from total_reserved
let total_reserved = amount
    .checked_add(max_reward)
    .ok_or(CustomError::Overflow)?
    .checked_add(total_ref_rewards)
    .ok_or(CustomError::Overflow)?;
    // market_fee intentionally excluded — already transferred to market
```

All stake calculations use pre-fee amount, inflating rewards, referral Payouts, and reservations while effectively bypassing the market fee

Resolved

Path

programs/vault/src/instructions/user/claim_withdrawal.rs

Description

When a user calls stake(), the full amount is deposited into the vault, then market_fee (5%) is immediately forwarded out to the market account. The net tokens the vault retains per stake are therefore amount - market_fee. However, every single downstream calculation — max_reward, total_ref_rewards, total_reserved, and the per-referrer accruals is computed against the pre-fee amount:

```
// Fee deducted from vault immediately
let market_fee = amount.checked_mul(5)?.checked_div(100)?;
spl_token::transfer(cpi_ctx_market, market_fee)?; // leaves vault

// But ALL calculations still use full `amount`
let max_reward       = calculate_max_reward(amount, period)?;  // should be amount - fee
let ref_reward       = amount.checked_mul(REF_REWARDS[level])?.checked_div(100)?; // should be post-fee
let total_reserved   = amount + max_reward + total_ref_rewards + market_fee; // all inflated
```

The Stake struct also records the full amount as the principal:

```
let stake = Stake {
    amount,          // stores pre-fee amount, not what vault actually holds
    ...
};
```

This creates a compounding set of errors:

1. **Staking rewards are inflated —** calculate_max_reward(amount, period) computes rewards on amount rather than amount - market_fee. The vault only ever holds amount - market_fee for this stake, yet promises to pay out rewards computed on the larger amount.
2. **Referral rewards are inflated —** each level's referral reward is amount * REF_REWARDS[level] / 100, again based on the full pre-fee amount. The vault is committed to paying more than its retained balance supports.
3. **`config.reserved` is further inflated —** total_reserved compounds all the above inflated values, making the solvency check progressively more optimistic than reality.
4. **Market fee is effectively bypassed —** because the Stake.amount field stores the pre-fee amount, when claim_stake_body is called at expiry it returns stake.amount + unclaimed_reward to the user, where both values were calculated on the pre-fee amount. The user effectively receives back the fee they were supposed to pay, at the expense of the vault's pre-loaded admin funds.

```
// claim_stake_body
let total_amount = stake.amount          // pre-fee amount returned to user
    .checked_add(unclaimed_reward)?;     // reward also on pre-fee amount
spl_token::transfer(cpi_ctx, total_amount)?;
```

Impact

1. **Fee bypass —** Users receive their full amount back at expiry plus rewards computed on amount, meaning the 5% market fee is ultimately borne by the admin's vault pre-load rather than the staker.
2. **Vault insolvency —** Rewards and referral payouts are promised on the pre-fee amount but the vault only retains amount - market_fee, causing the vault to be underfunded relative to its obligations at scale.
3. **Inflated reservations —** config.reserved grows faster than actual vault holdings, compounding the DoS issue from Issue (M -2) and accelerating the point at which all subsequent stakes are blocked.
4. **Unfair referral payouts —** Referrers receive rewards based on a staker's gross deposit rather than the net amount actually staked, draining the vault beyond its sustainable reward budget.

Remediation

Compute a net_amount after deducting the market fee and use it as the basis for all subsequent calculations, the Stake record, and config.reserved. Transfer the fee first so net_amount is well-defined before any calculation:

```rust
// Step 1: transfer user tokens into vault
spl_token::transfer(cpi_ctx, amount)?;

// Step 2: deduct and forward fee immediately
let market_fee = amount.checked_mul(5)?.checked_div(100)?;
spl_token::transfer(cpi_ctx_market, market_fee)?;

// Step 3: all further logic uses net_amount
let net_amount = amount.checked_sub(market_fee)?;

// Rewards based on net retained amount
let max_reward = calculate_max_reward(net_amount, period)?;

// Referral rewards based on net amount
let reward = net_amount
    .checked_mul(REF_REWARDS[level] as u64)?
    .checked_div(100)?;

// Stake records net amount as principal
let stake = Stake {
    amount: net_amount,  // what vault actually holds for this user
    ...
};

// Reserved only tracks what vault owes back
let total_reserved = net_amount
    .checked_add(max_reward)?
    .checked_add(total_ref_rewards)?;
// market_fee excluded — already left the vault
```

Malicious referees can grief their top level referrers by not providing their accounts in `remaining_accounts` and hence preventing them from receiving ref rewards

Resolved

Path

programs/vault/src/instructions/user/claim_withdrawal.rs

Description

In `stake()`, `total_ref_rewards` is computed by iterating up to 10 levels of the referral chain using only `current_referer` from the staker's `user_stats`. This loop does **not** walk the actual referral chain, it simply checks whether `current_referer.is_some()` and, because that value never changes inside the loop, it always iterates all 10 levels as long as any referrer exists at all:

```rust
```rust
let mut current_referer = user_stats.referer; // set once, never updated in loop

for level in 0..10 {
    if current_referer.is_some() {  // always true if user has any referrer
        let reward = amount
            .checked_mul(REF_REWARDS[level] as u64)?
            .checked_div(100)?;
        total_ref_rewards = total_ref_rewards.checked_add(reward)?;
        // current_referer is NEVER updated — loop always runs all 10 levels
    } else {
        break; // only breaks for admin (no referrer)
    }
}
```

This inflated total_ref_rewards is then included in total_reserved:

```rust
let total_reserved = amount
    .checked_add(max_reward)?
    .checked_add(total_ref_rewards)  // always 10-level worth, regardless of chain depth
    .checked_add(market_fee)?;

ctx.accounts.config.reserved = ctx.accounts.config.reserved
    .checked_add(total_reserved)?;   // vault locked for phantom referrers
```

Meanwhile, the actual accrual of rewards to referrers happens separately via remaining_accounts. If the caller provides zero or fewer than 10 referrer accounts, the loop over remaining_accounts simply breaks early:

```rust

    for level in 0..10 {
    let ref_link_info = match ctx.remaining_accounts.get(level) {
        Some(acc) => acc,
        None => break,  // silently exits — no referrer gets credited
    };
    // reward accrued to ref_link_info.ref_rewards[level]
}
```
```

This creates a critical divergence:

Scenario	total_ref_rewards added to reserved	Actual rewards credited to referrers
0 remaining_accounts provided	Full 10-level sum	0
3 remaining_accounts provided	Full 10-level sum	Only levels 0–2 credited
10 remaining_accounts provided	Full 10-level sum	All 10 levels credited (correct)

In the worst case (0 accounts provided), the vault permanently locks funds equal to the full 10-level referral reward sum, but no referrer ever receives or can claim those tokens. There is no mechanism to release or reclaim these phantom reservations.

Impact

1. **Permanent fund lockup :** Tokens equivalent to the full 10-level referral reward are permanently locked in config.reserved per stake where remaining_accounts is incomplete. These funds can never be claimed by any referrer (since their ref_rewards were never updated) and can never be released since there is no admin sweep or correction mechanism.
2. **Denial of Service on future stakes :** Because config.reserved is inflated by uncredited referral rewards on every under-specified stake call, the available_balance check deteriorates faster, causing legitimate future stakers to be rejected with InsufficientVaultBalance even when the vault holds sufficient real funds.
3. **Referrer reward starvation :** Any caller (malicious or simply careless) who omits remaining_accounts silently deprives the entire referral chain above the staker of their earned rewards with no error, no revert, and no recourse.
4. **Griefing vector :** A malicious staker can intentionally pass zero remaining_accounts on every stake to maximally inflate config.reserved and permanently grief the protocol at the cost of only their own staking deposit.

Remediation

Require that all referral accounts up to the full chain depth are always provided, and revert if any are missing:

```rust
// Require remaining_accounts covers the full referral chain
let mut expected_depth = 0;
let mut temp = user_stats.referer;
while temp.is_some() {
    expected_depth += 1;
    // walk on-chain to count true depth (up to 10)
    if expected_depth >= 10 { break; }
}
require!(
    ctx.remaining_accounts.len() >= expected_depth,
    CustomError::MissingReferralAccounts
);
```

break used instead of continue in claim_ref_rewards, permanently blocking higher-level reward claims

Resolved

Path

programs/vault/src/instructions/user/claim_withdrawal.rs

Description

In claim_ref_rewards(), the protocol iterates over all 10 referral levels and gates each level's reward behind a minimum active_referrals count requirement:

```
let required: [(usize, u64); 10] = [
    (0, 0),      // level 0: no requirement
    (0, 0),      // level 1: no requirement
    (0, 5),      // level 2: need 5 active referrals at level 0
    (1, 25),     // level 3: need 25 active referrals at level 1
    (2, 125),    // level 4: need 125 active referrals at level 2
    ...
];

for i in 0..10 {
    let (req_level, req_count) = required[i];
    if user_stats.active_referrals[req_level] < req_count {
        break;  // <-- exits the entire loop
    }
    ...
}
```

The break statement exits the entire loop the moment any single level's requirement is not met. This means if a referrer does not satisfy the requirement for level I`( which is very feasible given M-04), all rewards accumulated at levels `I+1 through 9 are silently skipped and can never be claimed in that call.

The intended behaviour is clearly to skip the current level and continue checking higher levels, since each level has independent requirements based on different req_level indices. A referrer can legitimately satisfy the requirement for level 4 (`active_referrals[2] >= 125`) without satisfying level 3 (`active_referrals[1] >= 25`), because these reference entirely different indices in active_referrals.

Concrete Scenario

Consider a referrer who:
• Has active_referrals[0] = 3 (below the level 2 requirement of 5)
• Has active_referrals[2] = 200 (above the level 4 requirement of 125)
• Has accumulated rewards at levels 4–9

When they call claim_ref_rewards():

```
level 0: req (0, 0) → passes, reward claimed
level 1: req (0, 0) → passes, reward claimed
level 2: req (0, 5) → active_referrals[0] = 3 < 5 → BREAK
```

The loop exits at level 2. All rewards at levels 3–9 are skipped entirely, even though the referrer fully qualifies for level 4 and above. Those rewards remain in user_stats.ref_rewards indefinitely but can never be claimed as long as active_referrals[0] stays below 5.

How `active_referrals` Gets Misaligned:
This is directly compounded by the issue described in (High-4). When stakers omit referrer accounts from remaining_accounts, the active_referrals counter at intermediate levels is never incremented. A referrer's account may have been included many times as a level-2 referrer (incrementing active_referrals[1]`) but never as a level-1 referrer (leaving `active_referrals[0] low), causing the break to fire early and permanently block all higher-level claims.

Impact

1. **Permanent reward loss :** Any referrer whose active_referrals count at any intermediate level falls below the threshold has all higher-level rewards permanently blocked due to the premature break.
2. **DoS on reward claims :** A malicious intermediary staker can deliberately omit accounts from remaining_accounts to keep a specific referrer's active_referrals[0] count low, permanently preventing that referrer from ever claiming levels 2–9 rewards regardless of how many deeper-level referrals they have.
3. **Incorrect reward accounting :** config.reserved still holds the reservation for these unclaimed rewards (from total_reserved in `stake()`), but the tokens can never be withdrawn, effectively locking them in the vault forever.

Remediation

Replace break with continue so each level is evaluated independently:

```
for i in 0..10 {
    let (req_level, req_count) = required[i];
    if user_stats.active_referrals[req_level] < req_count {
        continue;  // skip this level, keep checking higher levels
    }
    let reward = user_stats.ref_rewards[i];
    if reward == 0 {
        continue;
    }
    total = total
        .checked_add(reward)
        .ok_or(CustomError::Overflow)?;
    user_stats.ref_rewards[i] = 0;
}
```

Initialization can be frontrun, granting attacker full control over pre-funded vault

Resolved

Path

programs/vault/src/instructions/user/claim_withdrawal.rs

Description

The initialize() instruction accepts admin and market as arbitrary instruction parameters with no constraint tying them to the deployer or any trusted signer. The only guard against re-initialization is a check that config.admin != Pubkey::default(), which means whoever calls initialize() first wins:

```rust
// Anyone can call this — no signer constraint on who the admin must be
pub fn initialize(ctx: Context<Initialize>, market: Pubkey, admin: Pubkey) -> Result<()> {
    if config.admin != Pubkey::default() {
        return err!(CustomError::AlreadyInitialized); // only guard — first caller wins
    }
    config.admin = admin;   // attacker sets themselves as admin
    config.market = market; // attacker sets their own market account
}
```

A malicious actor can submit the transaction with admin and market with their own addresses, and submit with a higher priority fee to land first and hence take our program and hence the vault.

Attack Path

1. Team deploys the program and submits initialize() with their intended admin and market.
2. Attacker submits their own initialize() with admin = attacker_pubkey and market = attacker_token_account at higher priority.
3. Attacker's transaction lands first , config.admin and config.market are now set to attacker-controlled addresses.
4. Team's transaction lands and reverts with AlreadyInitialized.
5. The team pre-funds the vault (as required for the protocol to function — rewards must be pre-loaded by admin).
6. Attacker, now in control of config.admin, calls stake() as admin (admin can stake without a referrer), staking the minimum amount and draining the pre-funded vault balance over time via claim_stake_body() and claim_stake_rewards().
7. Additionally, all market fees from every legitimate user's stake flow to attacker_token_account indefinitely.

Impact

1. **Full vault takeover :** The vault is pre-funded by the team before users begin staking. With config.admin set to the attacker, the attacker can stake as admin and over time withdraw the entire pre-funded balance as staking rewards and principal.
2. **Protocol unusable for legitimate team :** Since AlreadyInitialized prevents re-initialization, the team has no on-chain recourse. The only recovery path is to upgrade and redeploy the program, abandoning any pre-funded vault balance.

Remediation

Remove the admin instruction parameter entirely and derive admin directly from the transaction signer. Since only the deployer will call initialize(), the payer/signer should become the admin by definition:

```
pub fn initialize(ctx: Context<Initialize>, market: Pubkey) -> Result<()> {
    let config = &mut ctx.accounts.config;
    if config.admin != Pubkey::default() {
        return err!(CustomError::AlreadyInitialized);
    }
    // Admin is always whoever signs the initialization — cannot be spoofed
    config.admin = ctx.accounts.payer.key();
    config.market = market;
    ...
}
```

Alternatively, if a specific admin address must be configurable, hardcode the expected deployer as a program constant and enforce it:

```
pub const EXPECTED_DEPLOYER: Pubkey = pubkey!("your_deployer_pubkey_here");

#[derive(Accounts)]
pub struct Initialize<'info> {
    #[account(
        mut,
        constraint = payer.key() == EXPECTED_DEPLOYER @ CustomError::NotAdmin
    )]
    pub payer: Signer<'info>,
    ...
}

```

Informational Issues

No admin transfer functionality

Resolved

Path

programs/vault/src/instructions/user/claim_withdrawal.rs

Description

The Config account stores an admin pubkey that gates privileged operations across the protocol. However, there is no instruction to transfer admin ownership to a new address. Once the program is initialized, the admin is permanently fixed.

```
pub struct Config {
    pub admin: Pubkey, // no way to ever change this
    ...
}
```

Impact

If the admin's private key is compromised, lost, or the team needs to rotate ownership (e.g., to a multisig), there is no on-chain mechanism to do so. The protocol becomes permanently bound to the original admin keypair with no recovery path.

Recommendation

Implement a two-step admin transfer pattern to prevent accidental transfers to wrong addresses:

```

pub fn propose_admin_transfer(ctx: Context<ProposeAdminTransfer>, new_admin: Pubkey) -> Result<()> {
    require!(ctx.accounts.admin.key() == ctx.accounts.config.admin, CustomError::NotAdmin);
    ctx.accounts.config.pending_admin = Some(new_admin);
    Ok(())
}

pub fn accept_admin_transfer(ctx: Context<AcceptAdminTransfer>) -> Result<()> {
    require!(
        Some(ctx.accounts.new_admin.key()) == ctx.accounts.config.pending_admin,
        CustomError::NotAdmin
    );
    ctx.accounts.config.admin = ctx.accounts.new_admin.key();
    ctx.accounts.config.pending_admin = None;
    Ok(())
}
```

No pause functionality

Resolved

Path

programs/vault/src/instructions/user/claim_withdrawal.rs

Description

The protocol has no emergency pause mechanism. All instructions — stake(), claim_stake_rewards(), claim_ref_rewards(), claim_stake_body() — execute unconditionally with no way for the admin to halt operations in the event of a discovered exploit, an ongoing attack, or a required upgrade.

Impact

In the event of a critical bug or active exploit, the team has no way to stop further damage. All user funds continue to be at risk until a program upgrade can be deployed

Remediation

Add a paused boolean flag to Config and gate all state-changing instructions behind it:

```
pub struct Config {
    ...
    pub paused: bool,
}

// At the top of every instruction:
require!(!ctx.accounts.config.paused, CustomError::ProtocolPaused);

// Add a toggle instruction gated to admin:
pub fn set_pause(ctx: Context<SetPause>, paused: bool) -> Result<()> {
    require!(ctx.accounts.admin.key() == ctx.accounts.config.admin, CustomError::NotAdmin);
    ctx.accounts.config.paused = paused;
    Ok(())
}

```

No functionality to change the market token account

Resolved

Path

programs/vault/src/instructions/user/claim_withdrawal.rs

Description

The market token account address is set once during initialize() and stored in Config. There is no instruction to update it afterward

```
config.market = market; // set once, never changeable
```

Impact

If the market account is compromised, deprecated, or needs to be rotated (e.g., migrating to a new treasury), the protocol has no recourse. All future market fees will continue flowing to the original address indefinitely.

Remediation

Add an admin-gated instruction to update the market account, with proper validation of the new account's mint:

```
pub fn update_market(ctx: Context<UpdateMarket>, new_market: Pubkey) -> Result<()> {
    require!(ctx.accounts.admin.key() == ctx.accounts.config.admin, CustomError::NotAdmin);
    // validate new_market is a token account with correct mint
    require!(
        ctx.accounts.new_market_account.mint == ctx.accounts.config.token_mint,
        CustomError::InvalidMint
    );
    ctx.accounts.config.market = new_market;
    Ok(())
}
```

Market token account not validated during initialization

Resolved

Path

programs/vault/src/instructions/user/claim_withdrawal.rs

Description

In initialize(), the market parameter is accepted as a raw Pubkey with no on-chain validation. The program never verifies that the provided address is an actual SPL token account, that its mint matches config.token_mint, or that it is owned by the token program.

```
pub fn initialize(ctx: Context<Initialize>, market: Pubkey, admin: Pubkey) -> Result<()> {
    ...
    config.market = market; // no validation whatsoever
}
```

Remediation

Pass market as a typed InterfaceAccount<TokenAccount> in the Initialize context with a mint constraint, rather than a raw Pubkey:

```
#[derive(Accounts)]
pub struct Initialize<'info> {
    ...
    #[account(
        constraint = market_account.mint == token_mint.key() @ CustomError::InvalidMint,
        constraint = market_account.owner == vault_authority.key() @ CustomError::InvalidMarketOwner
    )]
    pub market_account: InterfaceAccount<'info, TokenAccount>,
}

// In the instruction body:
config.market = ctx.accounts.market_account.key();
```

admin Parameter and admin_pubkey Account Can Differ During Initialization

Resolved

Path

programs/vault/src/instructions/user/claim_withdrawal.rs

Description

In initialize(), both an admin: Pubkey instruction parameter and an admin_pubkey: UncheckedAccount account are passed. The admin_stats PDA is derived from admin_pubkey, but config.admin is set from the admin parameter. There is no check that these two refer to the same address.

```
// admin_stats PDA derived from admin_pubkey account
seeds = [b"user_stats", admin_pubkey.key().as_ref()]

// but config.admin is set from the instruction parameter
config.admin = admin; // could be a completely different pubkey
```

Impact

The UserStats account created during initialization will belong to admin_pubkey, but the address recorded in config.admin may be a different pubkey. Any instruction that checks config.admin will mismatch with the account that actually has a UserStats initialized, potentially breaking admin staking or referral functionality.

Remediation

Add an explicit equality constraint or simply derive config.admin from admin_pubkey rather than accepting it as a separate parameter:

```
// Option A: Constraint in account struct
#[account(constraint = admin_pubkey.key() == admin @ CustomError::AdminMismatch)]
pub admin_pubkey: UncheckedAccount<'info>,

// Option B: Remove `admin` parameter, use admin_pubkey directly
config.admin = ctx.accounts.admin_pubkey.key();
```

admin Parameter and admin_pubkey Account Can Differ During Initialization

Resolved

Path

programs/vault/src/instructions/user/claim_withdrawal.rs

Description

The market fee rate is hardcoded as a magic number directly in the stake() instruction body with no configuration field and no admin instruction to update it.

```
// Hardcoded, no way to ever change this
let market_fee = amount
    .checked_mul(5)
    .ok_or(CustomError::Overflow)?
    .checked_div(100)
    .ok_or(CustomError::Overflow)?;
```

Impact

The protocol has no ability to adjust fee parameters in response to market conditions, governance decisions, or business requirements without a full program upgrade. This reduces the protocol's operational flexibility and makes even minor fee adjustments require redeployment.

Remediation

Store the fee rate in Config and expose an admin-gated instruction to update it, with a reasonable cap to protect users:

```
pub struct Config {
    ...
    pub market_fee_bps: u16, // basis points, e.g. 500 = 5%
}

// In stake():
let market_fee = amount
    .checked_mul(ctx.accounts.config.market_fee_bps as u64)?
    .checked_div(10_000)?;

// Admin instruction:
pub fn update_fee(ctx: Context<UpdateFee>, new_fee_bps: u16) -> Result<()> {
    require!(ctx.accounts.admin.key() == ctx.accounts.config.admin, CustomError::NotAdmin);
    require!(new_fee_bps <= 1000, CustomError::FeeTooHigh); // max 10%
    ctx.accounts.config.market_fee_bps = new_fee_bps;
    Ok(())
}
```

initialize() derives PDA bumps via find_program_address instead of using ctx.bumps

Resolved

Path

programs/vault/src/instructions/user/claim_withdrawal.rs

Description

In initialize(), the bumps for vault and vault_authority PDAs are re-derived using Pubkey::find_program_address, which is a computationally expensive off-chain search performed on-chain. Anchor already computes and exposes these bumps in ctx.bumps during account validation for free.

```
// Expensive re-derivation — already done by Anchor
let (_, vault_bump) = Pubkey::find_program_address(
    &[b"vault", config.key().as_ref()],
    ctx.program_id
);
let (_, vault_authority_bump) = Pubkey::find_program_address(
    &[b"vault_authority", config.key().as_ref()],
    ctx.program_id
);
```

Impact

Each find_program_address call consumes significant compute units (CU) by iterating through nonces until a valid off-curve point is found. This wastes CU budget unnecessarily, increases transaction cost, and risks hitting compute limits on complex transactions.

Remediation

Use the bumps already computed by Anchor's account validation, available through ctx.bumps:

```

config.vault_bump = ctx.bumps.vault;
config.vault_authority_bump = ctx.bumps.vault_authority;

```

AddReferralCtx does not explicitly validate referral_token_account mint against config.token_mint

Resolved

Path

programs/vault/src/instructions/user/claim_withdrawal.rs

Description

In the AddReferralCtx account struct, the referral_token_account is initialized with associated_token::mint = token_mint, where token_mint is a separate account passed in the context. However, there is no constraint verifying that token_mint matches config.token_mint. A caller can pass a different mint account, causing the referral token account to be initialized for the wrong token and the referral transfer to credit the wrong token to the referee.

```
1
2  #[account(
3      init_if_needed,
4      payer = referer,
5      associated_token::mint = token_mint, // token_mint is not validated against config
6      associated_token::authority = referral,
7      associated_token::token_program = token_program
8  )]
9  pub referral_token_account: InterfaceAccount<'info, TokenAccount>,
10
11 pub token_mint: InterfaceAccount<'info, Mint>, // no constraint: token_mint == config.token_mint
12
```

However, the transaction would fail because the sender's mint is checked against config.mint in case the wrong mint is passed, it's always best practice to validate mints.

Remediation

Add an explicit constraint on token_mint in AddReferralCtx to enforce it matches the mint stored in config:

```
1  #[account(
2      constraint = token_mint.key() == config.token_mint @ CustomError::InvalidMint
3  )]
4  pub token_mint: InterfaceAccount<'info, Mint>,
5
6
7  #[account(
8      mut,
9      constraint = referer_token_account.mint == config.token_mint @ CustomError::InvalidMint,
10 )]
11 pub referer_token_account: InterfaceAccount<'info, TokenAccount>,
12
```

transfer used instead of transfer_checked on a Token-2022 mint

Resolved

Path

programs/vault/src/instructions/user/claim_withdrawal.rs

Description

The protocol uses token_2022 aliased as spl_token and calls the basic transfer instruction for all token movements:

```
use anchor_spl::token_2022 as spl_token;

spl_token::transfer(cpi_ctx, amount)?;
spl_token::transfer(cpi_ctx_market, market_fee)?;
spl_token::transfer(cpi_ctx_ref, reward)?;
```

Currently, the mint only has the token metadata extension enabled, which does not affect transfer behaviour and is compatible with the basic transfer call. So in the current deployment, this works without issue.

However, the mint is a Token-2022 mint, and Token-2022 was specifically designed to support extensions such as transfer fees, transfer hooks, confidential transfers, and more. These extensions are only properly triggered and enforced when using transfer_checked. The basic transfer instruction silently bypasses all of them.

Additionally, transfer is officially deprecated in Token-2022 by the Solana token program in favour of transfer_checked. Continuing to use the deprecated instruction is against best practices for Token-2022 programs.

Remediation

Replace all spl_token::transfer calls with transfer_checked as a proactive best practice:

```rust
// Instead of:
spl_token::transfer(cpi_ctx, amount)?;

// Use:
spl_token::transfer_checked(
    cpi_ctx,
    amount,
    ctx.accounts.token_mint.decimals,
)?;

// Updated CPI accounts struct:
let cpi_accounts = spl_token::TransferChecked {
    from: ctx.accounts.user_token_account.to_account_info(),
    mint: ctx.accounts.token_mint.to_account_info(), // additionally required
    to: ctx.accounts.vault.to_account_info(),
    authority: ctx.accounts.user.to_account_info(),
};
```

This ensures the protocol remains forward-compatible with any future mint extension upgrades and aligns with Token-2022 best practices.

Closing Summary

In this report, we have considered the security of the LifeSol Solana Programs. We performed our audit according to the procedure described above.

Some issues of Critical, Medium and informational severity were found, Some suggestions and best practices are also provided in order to improve the code quality and security posture.

Disclaimer

QuillAudits Smart contract security audit provides services to help identify and mitigate potential security risks in Lifesol Solana Programs smart contract. However, it is important to understand that no security audit can guarantee complete protection against all possible security threats. QuillAudits audit reports are based on the information provided to us at the time of the audit, and we cannot guarantee the accuracy or completeness of this information. Additionally, the security landscape is constantly evolving, and new security threats may emerge after the audit has been completed.

Therefore, it is recommended that multiple audits and bug bounty programs be conducted to ensure the ongoing security of LifeSol Solana Programs smart contracts. One audit is not enough to guarantee complete protection against all possible security threats. It is important to implement proper risk management strategies and stay vigilant in monitoring your smart contracts for potential security risks.

QuillAudits cannot be held liable for any security breaches or losses that may occur subsequent to and despite using our audit services. It is the responsibility of the LifeSol Solana Programs to implement the recommendations provided in our audit reports and to take appropriate steps to mitigate potential security risks.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

March 2026

For

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com